

Relazione progetto IA

Sebastiano Patton

2026-05-25

Introduzione

Lo scopo del progetto è quello di creare un'intelligenza artificiale in grado di svolgere una semplice task e confrontarla con un'altra IA pre-addestrata. La task che ho scelto consiste nel separare le immagini in due categorie.

Data set e preprocessing

Come dataset ho scelto: Chest X-Ray Images (Pneumonia), ovvero un dataset su kaggle.com contenente immagini di radiografie di pazienti sani e pazienti malati di polmonite. L'intelligenza artificiale deve riconoscere quali radiografie sono di pazienti malati e quali non lo sono. Le differenze tra le due tipologie di immagini sono tipicamente:

- Pazienti sani: la radiografia mostra chiaramente le ossa e i polmoni
- Pazienti malati: la radiografia presenta zone offuscate nella parte alta o nella parte bassa e i polmoni sono abbastanza sfuocati.

Ho scelto questo dataset in quanto forniva una cartella con un buon numero di immagini per il training e un'altra cartella con delle immagini per il test finale per verificare l'efficacia della rete neurale dopo il training. Un dettaglio da notare è che nella cartella di training sono presenti circa il triplo delle immagini di pazienti malati rispetto ai pazienti sani il che potrebbe creare problemi con il training.

Per rendere adatte le foto al training bisogna prima fare un preprocessing. Siccome la rete neurale che utilizziamo è una rete CNN necessita che le immagini in ingresso siano di dimensioni equivalenti per poterle raggruppare in pacchetti (batch), per questo nel codice utilizzo il comando `transform.resize((150, 150))` che permette di ridimensionare tutte le immagini ad una risoluzione 150x150, sufficiente per il training. Ho anche aumentato "artificialmente" la quantità di immagini aggiungendo `transforms.RandomHorizontalFlip()`, ovvero un comando che ha una chance del 50% di invertire un'immagine andando effettivamente a crearne una nuova per la rete neurale in modo da aumentare la robustezza e prevenire l'overfitting. Per evitare problemi di "gradient explosion", ovvero che i calcoli matematici interni alla rete vadano in saturazione, converto i valori dei pixel da 0 a 255 in float che vanno da 0.0 a 1.0. Per rendere le nostre immagini adatte al modello di IA importato (MobileNetV2) ho anche configurato le immagini in modo che abbiano la stessa distribuzione statistica di quello su cui è addestrata la IA importata ovvero `mean=[0.485, 0.456, 0.406]` e `std=[0.229, 0.224, 0.225]` in quanto la rete MobileNetV2 è pre-addestrata su pixel con l'intervallo -

2,2 e con quella specifica distribuzione e andrebbe a funzionare in modo caotico se le presentassi immagini che non rispettano quei parametri.

```
# Trasformazioni
data_transforms = {
    'train': transforms.Compose([
        transforms.Resize((150, 150)),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'val': transforms.Compose([
        transforms.Resize((150, 150)),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
}

# Percorsi (adattati alla tua cartella)
train_dir = './chest_xray/train'
val_dir = './chest_xray/test'

train_data = datasets.ImageFolder(train_dir, transform=data_transforms['train'])
val_data = datasets.ImageFolder(val_dir, transform=data_transforms['val'])

train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
val_loader = DataLoader(val_data, batch_size=32, shuffle=False)

print(f"Immagini di training: {len(train_data)}")
print(f"Classi rilevate: {train_data.classes}")
```

Setup ambiente e hardware

Per consentire la riproducibilità del progetto uso un ambiente virtuale venv, così facendo il resto del sistema operativo non andrà ad influire sui risultati. Ho dovuto adottare dei accorgimenti nel codice in quanto il mio computer ha una GPU AMD, quindi ho dovuto utilizzare DirectML al posto di CUDA che invece funziona su gpu NVIDIA. Quindi PyTorch invia i calcoli al pacchetto pytorch-dyrectml che li traduce sfruttando le librerie DirectX 12 che interagiscono direttamente con la gpu AMD andando a sfruttare la sua potenza di calcolo. Per scrivere il programma ho utilizzato un Jupyter notebook in quanto la sua architettura a celle permette di tenere tutto il codice insieme e evita continui caricamenti del dataset in memoria permettendo un'esecuzione modulare, semplificando anche il troubleshooting in quanto sarebbe stato ristretto nelle celle in cui avviene l'errore.

Sviluppo delle architetture

Modello importato: MobileNetV2

Ho scelto la rete neurale MobileNetV2 in quanto è stata addestrata su più di un milione di immagini allenata apposta per riconoscere immagini in 1000 classi oggetto diverse. Questa rete è idealmente divisibile in due parti:

- Feature extractor: decine di strati convoluzionali che analizzano l'immagine e estraggono i dettagli visivi (come linee, sfumature, punti di luce...)
- Classifier: l'ultimo strato della rete che assegna ad ogni immagine una classe a cui secondo lui appartiene

Per il mio progetto non servono 1000 classi oggetto diverse, me ne bastano due ovvero: NORMAL e PNEUMONIA, quindi ho apportato delle lievi modifiche MobileNetV2. Ho mantenuto la parte di feature extraction in quanto sarebbe stato in grado di riconoscere i vari dettagli delle immagini. Per farlo ho bloccato tutti i pesi pre-addestrati (for param in model.parameters(): param.requires_grad = False), in questo modo congelo i pesi ed evito che i gradienti modifichino le feature ereditate da MobileNetV2, riducendo anche i carichi di calcolo. Ho poi eliminato lo strato Classifier con 1000 nodi di uscita e lo ho sostituito con uno nuovo con solo 2 nodi di uscita (model.classifier[1] = nn.Linear(model.last_channel, 2)), così facendo il nuovo strato Classifier trasmuterà tutti i dati dei nodi in ingresso in soli due output. Ovviamente i nodi del nuovo classifier andranno addestrati; per questo motivo hanno il parametro requires_grad=True attivo di default.

```
# Carico il modello pre-addestrato
model = models.mobilenet_v2(weights='DEFAULT')

# Congelo i parametri per il Transfer Learning
for param in model.parameters():
    param.requires_grad = False

# Modifico il classificatore finale
model.classifier[1] = nn.Linear(model.last_channel, 2)

model = model.to(device)
print("Modello configurato e inviato alla GPU AMD.")
```

Modello LeNet-5

Ho deciso di usare un modello LeNet-5 dato che è un modello semplice da implementare.

Il modello LeNet è formato da due strati convoluzionali: conv1 con 3 canali di input, 6 filtri, kernel 5x5 che riconosce le forme geometriche di base e conv2 con 6 canali in ingresso, 16 filtri, kernel 5x5 che unisce le figure geometriche per creare i forme complesse. Ha poi 2 strati di pooling che entrambi dimezzano le dimensioni dell'immagine portando da 150x150 pixel a 34x34. I livelli fully connected fc1 fc2 che sono rispettivamente gli strati densi da 120 e da 84 neuroni e infine fc3 con 2 neuroni per l'output. Infine c'è self.activation = nn.Tanh() caratteristico di LeNet-5 che limita tutti i valori tra -1 e 1 in modo che i dati siano semplici bilanciati intorno a zero

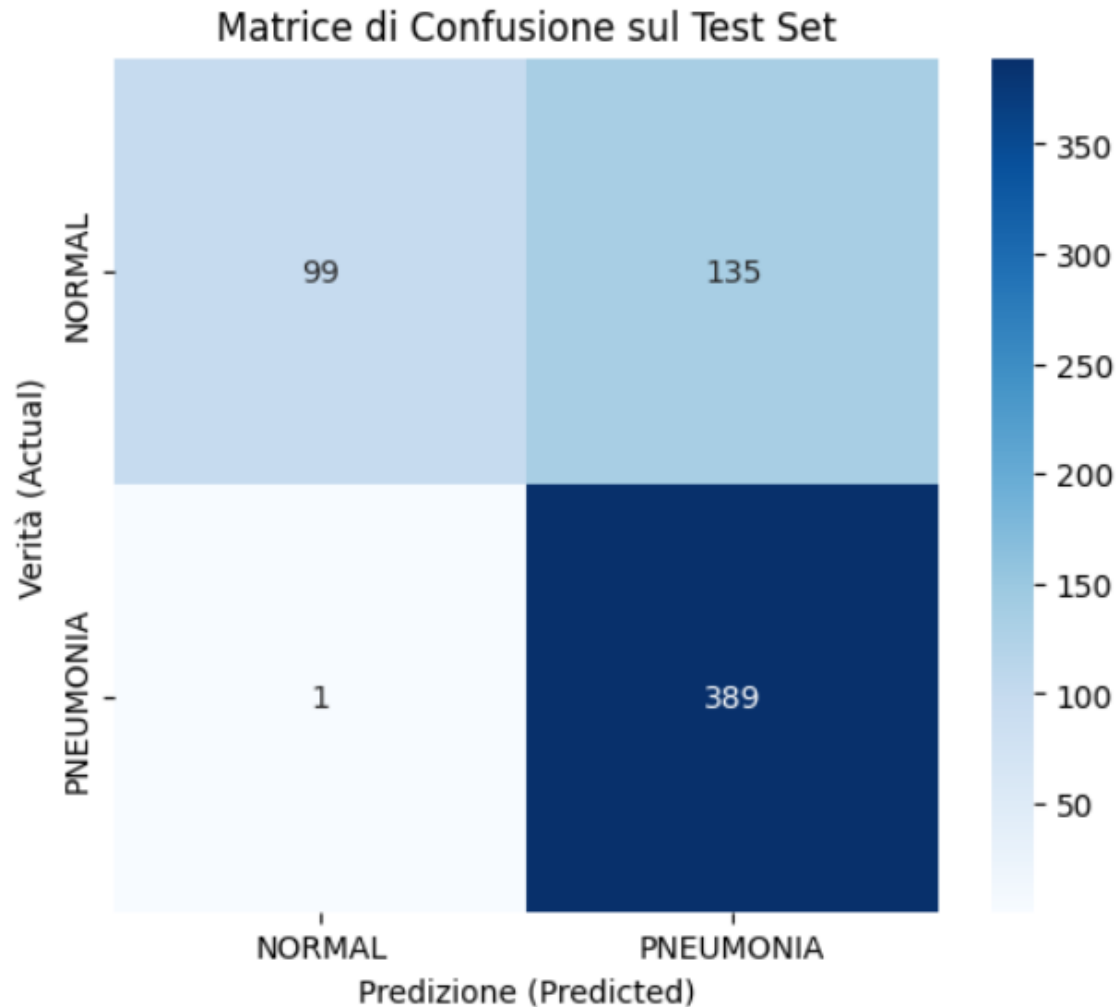
Esperimenti e ottimizzazione

MobileNetV2

Sono partito facendo il training della rete MobileNetV2, il primo training lo ho fatto durare 10 epoche con ottimizzatore Adam, learning rate pari a 10^{-4} e una funzione di costo classica di entropia incrociata con penalizzazione uguale su entrambe le classi. Le cose che ho notato sono due:

- 1) inizialmente rapida minimizzazione della loss e diminuzione di loss fino a risultati intorno al 0,12/0,13 nelle ultime 3 epoche
- 2) Nelle ultime epoche i valori di precisione e di loss miglioravano veramente poco

Per questo motivo ho scelto di diminuire il numero di epoche nel training a 7 in modo da diminuire i tempi di training da 18-19 minuti a 12-13 minuti mantenendo una loss alla fine del training di circa 0,16. Ho quindi eseguito il test per verificare se avrebbe funzionato bene e qui è dove ho riscontrato il problema dello sbilanciamento del dataset di training. Come detto all'inizio il dataset di training contiene circa 1/4 delle immagini sono di pazienti sani e i restanti 3/4 sono di pazienti malati, mentre nel dataset per il test le immagini di malati e sani sono circa 50/50. Per questo motivo la rete è stata vittima di un majority bias che ha fatto polarizzare i filtri verso la classe maggiore facendo sì che per ottimizzare scommettesse sulla presenza di polmonite per ridurre l'errore medio. Ciò è evidenziato dalla matrice di confusione



in cui si notano ben 135 falsi positivi di polmonite che sarebbero dovuti risultare sani.

Per prevenire questo problema sono andato a modificare la funzione di costo creando un vettore di pesi asimmetrico dove ho assegnato un costo maggiore alla classe dei pazienti sani rispetto a quelli malati così facendo moltiplico del costo l'errore di quando indovina un falso positivo. Ho fatto 5 test aumentando il costo della classe sana e controllando la precisione del test finali:

TABELLA PRECISIONE IN BASE AL COSTO

HO deciso quindi di tenere un costo di 6 come definitivo per alzare l'accuratezza del test da 70% a 86%.

```
import torch

# 1. Configurazione dei Pesi (Diamo peso 6.0 ai sani e 1.0 ai malati)
class_weights = torch.tensor([6.0, 1.0]).to(device)
criterion = nn.CrossEntropyLoss(weight=class_weights)

# 2. Abbassiamo il Learning Rate a 0.0005 per evitare balzi troppo bruschi
optimizer = optim.Adam(model.parameters(), lr=0.0005)

# Reset delle liste per i grafici
train_losses = []
train_accuracies = []
```

LeNet-5

Dati i risultati del training della MobileNetV2 ho immediatamente impostato un vettore pesi asimmetrico con un costo iniziale di 3.0 : 1.0 e un totale di 10 epoche. Ho aumentato il numero di epoche e abbassato il costo da 6 a 3 perchè è un modello meno potente di MobileNetV2 e quindi credo che necessiti di più epoche per ottenere un training più adatto.

```
Inizio Training Modello 2 (LeNet-5 Adattata)...
c:\Progetto_ia\venv\Lib\site-packages\torch\optim\adam.py:5
  torch._foreach_lerp_(device_exp_avgs, device_grads, 1 - b
LeNet-5 - Epoca 1/10 - Loss: 0.2525 - Accuracy: 89.55%
LeNet-5 - Epoca 2/10 - Loss: 0.1807 - Accuracy: 92.45%
LeNet-5 - Epoca 3/10 - Loss: 0.1610 - Accuracy: 93.50%
LeNet-5 - Epoca 4/10 - Loss: 0.1435 - Accuracy: 94.13%
LeNet-5 - Epoca 5/10 - Loss: 0.1217 - Accuracy: 94.96%
LeNet-5 - Epoca 6/10 - Loss: 0.1175 - Accuracy: 95.25%
LeNet-5 - Epoca 7/10 - Loss: 0.1072 - Accuracy: 95.74%
LeNet-5 - Epoca 8/10 - Loss: 0.0987 - Accuracy: 96.20%
LeNet-5 - Epoca 9/10 - Loss: 0.0926 - Accuracy: 96.40%
LeNet-5 - Epoca 10/10 - Loss: 0.0845 - Accuracy: 96.68%
Addestramento LeNet-5 completato con successo!
Accuratezza finale della LeNet-5 sul Test Set: 74.20%
```

con un costo di 3:0 nel training ho avuto una loss di 0,08 e sul test un accuracy di 74%, ho quindi deciso di controllare se sarebbe migliorato andando ad aumentare il costo a 6 come per la MobileNetV2 e così è stato

```

Inizio Training Modello 2 (LeNet-5 Adattata)...
LeNet-5 - Epoca 1/10 - Loss: 0.2446 - Accuracy: 84.07%
LeNet-5 - Epoca 2/10 - Loss: 0.1702 - Accuracy: 90.70%
LeNet-5 - Epoca 3/10 - Loss: 0.1352 - Accuracy: 92.37%
LeNet-5 - Epoca 4/10 - Loss: 0.1240 - Accuracy: 93.35%
LeNet-5 - Epoca 5/10 - Loss: 0.1180 - Accuracy: 93.35%
LeNet-5 - Epoca 6/10 - Loss: 0.1031 - Accuracy: 94.42%
LeNet-5 - Epoca 7/10 - Loss: 0.0899 - Accuracy: 95.38%
LeNet-5 - Epoca 8/10 - Loss: 0.0874 - Accuracy: 95.71%
LeNet-5 - Epoca 9/10 - Loss: 0.0732 - Accuracy: 96.01%
LeNet-5 - Epoca 10/10 - Loss: 0.0741 - Accuracy: 96.30%
Addestramento LeNet-5 completato con successo!

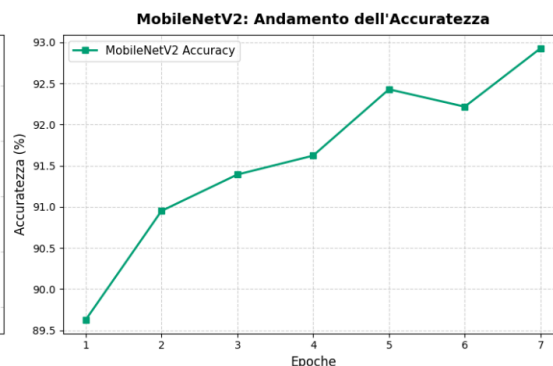
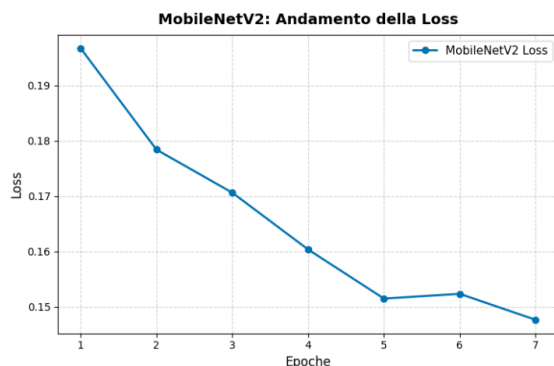
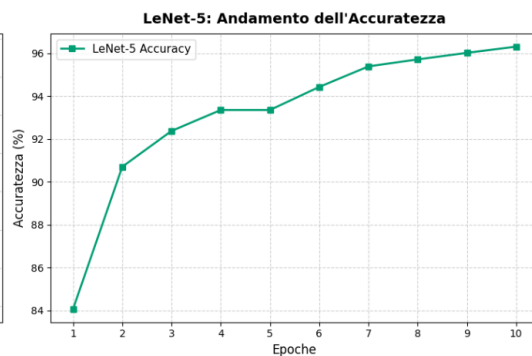
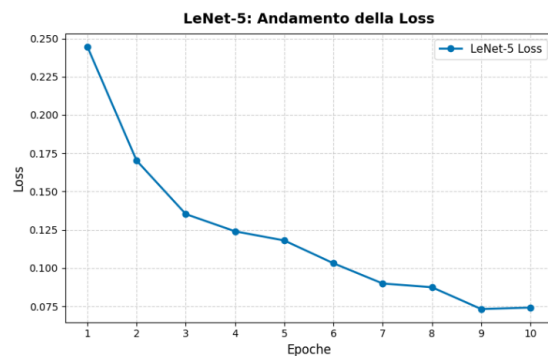
```

Accuratezza finale della LeNet-5 sul Test Set: 79.65%

così facendo

la loss è scesa a 0,07 e l'accuracy del test è salita a 79%. Ho provato ad aumentare ulteriormente il costo portandolo a dieci ma ha solo aumentato la loss a 0,06 e andando ad diminuire la precisione a 77% aumentando il numero sia di falsi negativi che di falsi positivi anche se di poco

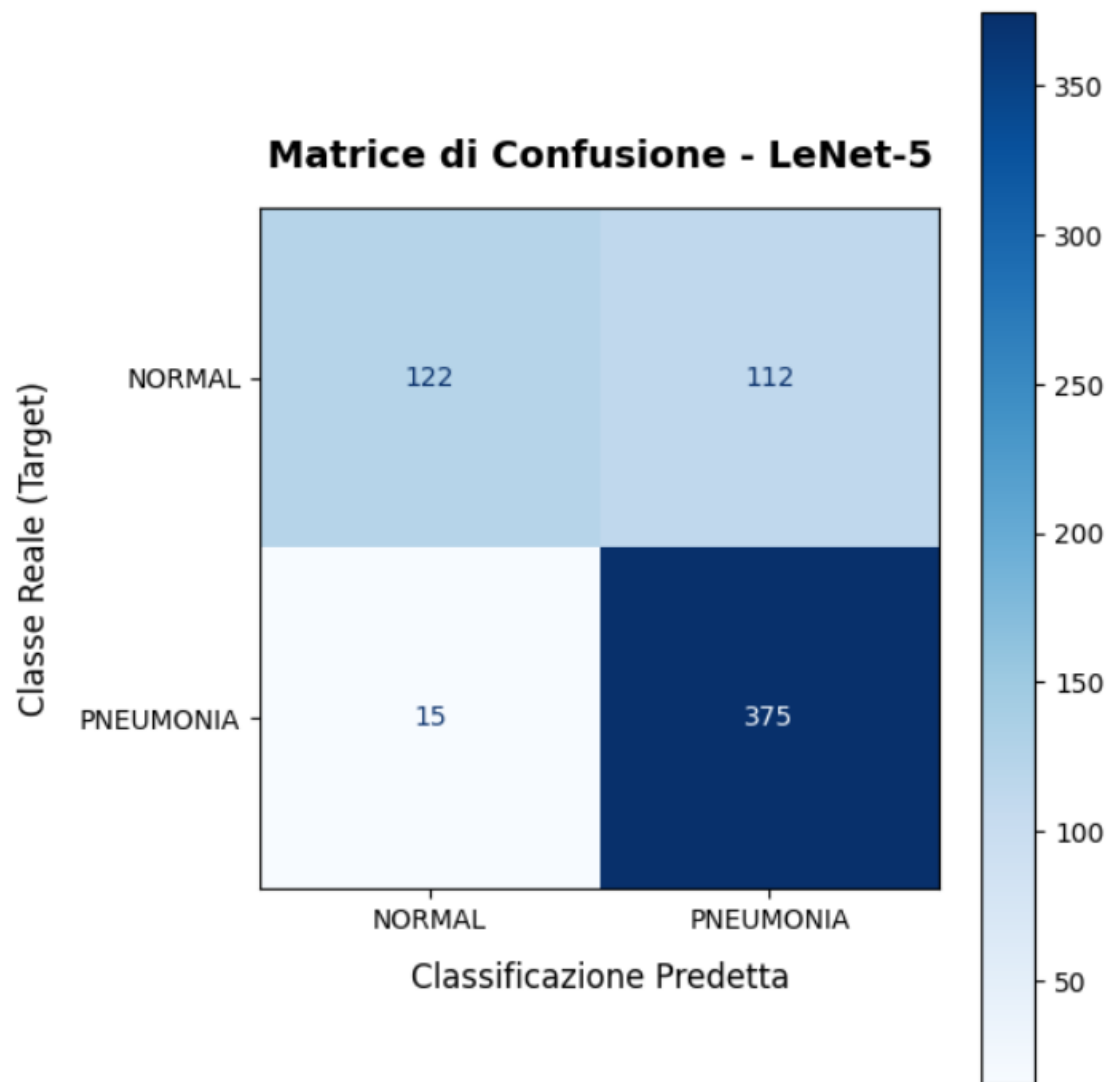
Analisi comparativa dei risultati

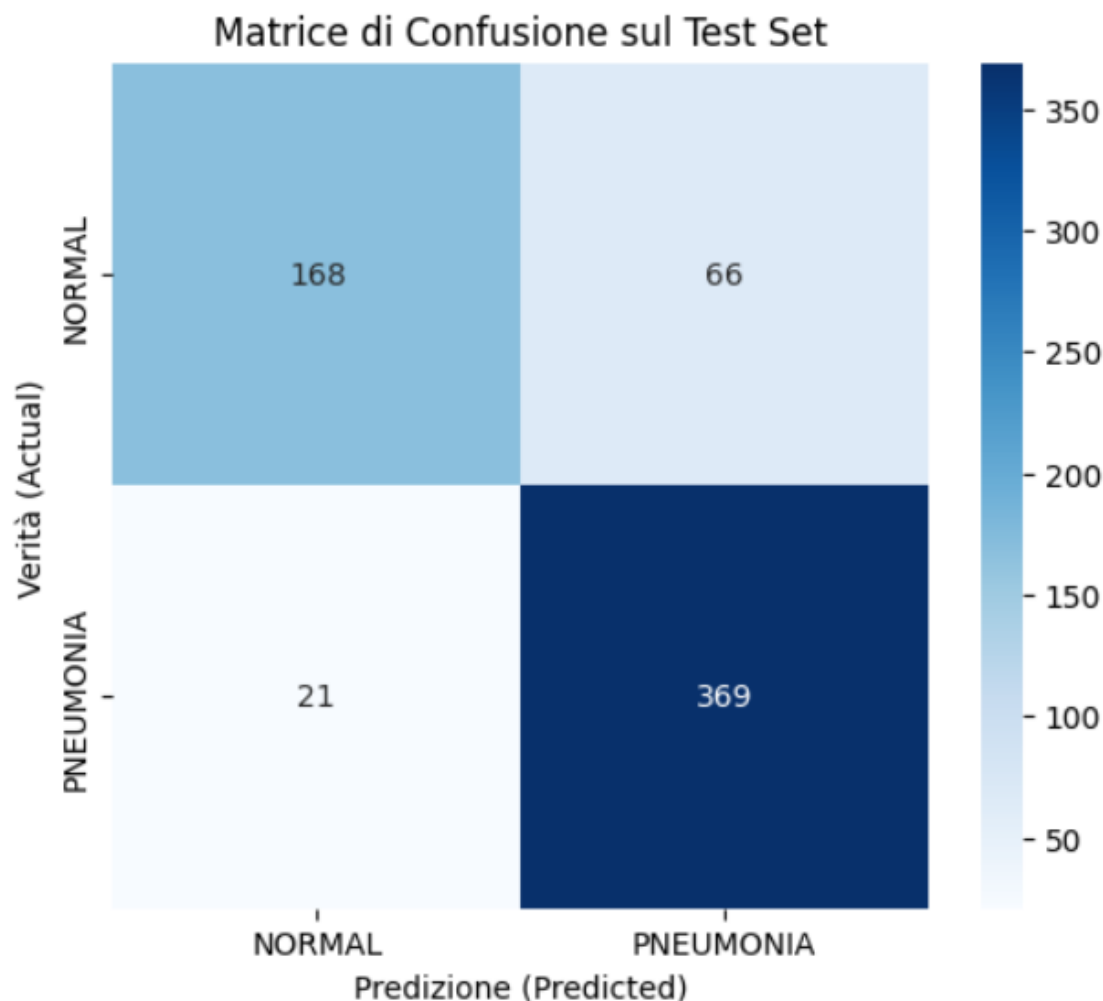


Nelle

immagini che mostrano i tempi di training l'accuracy è un calcolo puramente statistico in cui confronta la decisione fatta con l'etichetta esatta del test e lo uso semplicemente per vedere quante immagini corrette indovina, il dato più importante rimane la loss che indica l'errore della rete nel fare le decisioni

Partendo dall'analisi del training, grafici si nota chiaramente che la MobileNetV2 ha una loss che converge rapidamente , ciò è evidenziato dal numero ridotte di epoche, grazie ai pesi pre addestrati. La LeNet-5 ha una andamento abbastanza lineare raggiungendo dei valori di loss estremamente bassi.





Confrontando le matrici di confusione è evidente che MobileNetV2 riesci a separare chiaramente le due classi mentre la Lenet ha un forte sbilanciamento verso la classe PNEUMONIA ovvero quella maggioritaria avendo un gran numero di falsi negativi. È evidente che la MobileNetV2 è superiore ottenendo un accuracy sul test finale di 86% in confronto al 79% della Lenet, il motivo credo sia dato dall'overfitting in quanto la LeNet-5 era pensata per analizzare immagini di 32x32 pixel quindi quando li si presentano di 150x150 pixel si vanno a creare troppi parametri che permettono alla rete di avere più di 2.2 milioni di pesi che negli ultimi strati della LeNet vengono usati per memorizzare le immagini, non imparare i pattern, questo causa un enorme drop nella precisione con il test finale.

Conclusione

Ritengo che per il dataset che ho analizzato il costo ideale sia 6 : 1 dato lo sbilanciamento dei dati e sceglierei la rete MobileNetV2 in quanto si è dimostrata superiore alla LeNet-5 ottenendo risultati migliori nel test con meno tempo di training.